

# Evaluation - E 5

## Réaliser une application intégrant un service d'intelligence artificielle - monitoring et la gestion des incidents

### Sommaire

- [C20. Surveiller une application d'intelligence artificielle](#) en mobilisant des techniques de monitoring et de journalisation, dans le respect des normes de gestion des données personnelles en vigueur, afin d'alimenter la feedback loop\* dans une approche MLOps, et de permettre la détection automatique d'incidents.
- [C21. Résoudre les incidents techniques](#) en apportant les modifications nécessaires au code de l'application et en documentant les solutions pour en garantir le fonctionnement opérationnel.

---

### Documentation de l'Observabilité et Gestion des Incidents (MCO)

#### Projet : Waterflow 2 (Drink Safe)

Ce document synthétise la stratégie de monitoring applicatif et la méthodologie de résolution d'incidents mises en œuvre sur la plateforme Drink Safe, conformément aux attentes MLOps et aux exigences de Maintien en Condition Opérationnelle (MCO). Cette documentation a été formatée en respectant les recommandations d'accessibilité numérique de l'association [Valentin Haüy](#) et du [RGAA](#) (hiérarchie sémantique des titres, contrastes visuels clairs)

---

## C20. Surveiller une application d'intelligence artificielle

Pour garantir la haute disponibilité du service et la détection proactive des anomalies, la plateforme s'appuie sur une stack d'observabilité complète et découplée.

### 1. Architecture de Supervision (Stack Prometheus / Grafana)

Le monitoring repose sur trois composants aux rôles stricts :

- **Exposition (FastAPI)** : L'API Unique expose un point de terminaison `/metrics` via la librairie `prometheus_client`.
- **Stockage Time-Series (Prometheus)** : Un conteneur Prometheus (port 9090) "scrape" (récupère) ces métriques à intervalles réguliers (toutes les 15 secondes).
- **Visualisation (Grafana)** : Un tableau de bord Grafana (port 3000) interroge Prometheus en PromQL pour afficher l'état de santé du système.

### 2. Le Tableau de Bord "RED"

Le suivi en temps réel pour le Responsable d'Exploitation repose sur la méthode RED, qui trace les trois indicateurs vitaux d'une API :

- **Rate (Trafic)** : Le volume de requêtes HTTP par seconde (`http_requests_total`).
- **Errors (Fiabilité)** : Le pourcentage d'erreurs serveur 5xx générées par le backend.
- **Duration (Performance)** : La latence des requêtes, calculée au 95ème centile pour identifier les goulots d'étranglement.

### 3. Audit Trail et Journalisation Structurée

- **Logs JSON structurés** : Pour faciliter le diagnostic, l'application génère des logs au format JSON contenant le contexte de l'événement (ID client, endpoint, durée, erreur exacte). Ce format évite les "prints" sauvages impossibles à filtrer.
  - **Middleware d'Audit (Base de données)** : Un middleware HTTP intercepte chaque requête, anonymise la clé API pour des raisons de sécurité, et consigne l'action (date, route, code statut, durée) dans la table PostgreSQL `action_logs`. Cela permet de répondre aux audits RGPD tout en surveillant l'utilisation de l'API.
  - **Métriques métiers (Custom)** : Le système incrémente des compteurs spécifiques, comme `ocr_failures_total`, pour tracer spécifiquement les pannes du service d'extraction documentaire externe sans noyer les logs globaux.
-

## C21. Résoudre les incidents techniques

Face aux pannes et aux bogues, l'équipe applique la méthodologie stricte **DDCR** : Détection, Diagnostic, Correction, Retour d'expérience.

**Incident #001 - Panne du service externe OCR.space**

**Sévérité** : Majeure (Fonctionnalité d'ingestion OCR totalement bloquée, entraînant des erreurs 504 puis un crash de l'interface client).

### 1. Détection

- **Détection initiale** : Crash de l'interface utilisateur lors du téléversement d'une fiche laboratoire.
- **Détection Monitoring** : Le compteur Prometheus `ocr_failures_total` s'incrémente. Une alerte Grafana a été déclenchée sur la métrique `rate(http_requests_total{endpoint="/api/ocr/lab-report",status=~"5.."}[5m]) > 0.1`.

### 2. Diagnostic

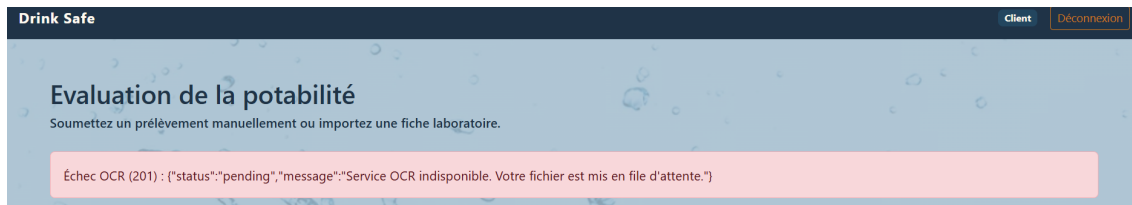
- **Panne du service tiers** : L'API distante `api.ocr.space` ne répond plus dans le temps imparti (Timeout > 15s) ou renvoie une erreur (Bad Gateway).
- **Impact API (Backend)** : L'API FastAPI n'était pas résiliente. Elle levait une `HTTPException` directe faisant échouer l'appel.
- **Impact Frontend (Flask)** : Même après avoir ajouté un message de secours côté API, le client Flask s'attendait strictement à recevoir des mesures physico-chimiques. Ne les trouvant pas, le template HTML a déclenché une erreur fatale Jinja2 (`TypeError: Object of type Undefined is not JSON serializable`).

### 3. Correction

Mise en place d'une dégradation gracieuse du service (Fallback) en deux étapes :

- **Côté API (`src/routes/ocr.py`)** : Ajout d'un bloc `try/except` interceptant le Timeout. L'API ne crashe plus et renvoie désormais un statut pending (HTTP 201) accompagné d'un message d'attente.
- **Côté Frontend (`front/client.py`)** : Ajout d'une condition pour intercepter la réponse JSON `{"status": "pending"}`. Le client est désormais redirigé proprement vers son tableau de bord avec une alerte visuelle jaune (`flash("warning")`) expliquant que le service est ralenti, évitant ainsi le crash de la page HTML.

Le client Flask évite le crash grâce au Fallback gracieux :



l'API retourne un code 201 Pending au lieu d'une erreur 500 :

| Request URL                                           |                                                                                                                                              |
|-------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <code>http://127.0.0.1:8000/api/ocr/lab-report</code> |                                                                                                                                              |
| Server response                                       |                                                                                                                                              |
| Code                                                  | Details                                                                                                                                      |
| 201                                                   | <div>Response body<pre>{  "status": "pending",  "message": "Service OCR indisponible. Votre fichier est mis en file d'attente."}</pre></div> |

#### 4. Prévention et Retour d'Expérience (MCO)

- **Visibilité** : Remplacement des exceptions fatales et des `print()` par des **logs structurés au format JSON** (`logger.error("ocr_call_failed", ...)`), incluant le `client_id` et la durée de la panne (`duration_ms`), permettant un diagnostic rapide en exploitation.
- **Résilience** : Le système est désormais découplé de la santé de l'OCR externe. L'application continue de fonctionner pour les autres routes (prédiction standard, consultation) même si l'OCR est hors-service.
- **Amélioration future** : Mettre en place un système de file d'attente (retry asynchrone) pour traiter automatiquement le document dès que l'OCR.space sera de nouveau en ligne.

#### Registre de Maintien en Condition Opérationnelle (MCO)

D'autres incidents critiques liés à l'architecture et à la sécurité ont été diagnostiqués, fixés, testés, puis consignés dans le registre global du projet (`bugfix.md`). Parmi les cas résolus :

- **Désynchronisation MLOps (Crash Inférence 503)** : Résolu par la mise en place d'un chargement dynamique (Lazy Loading) et le partage des volumes Docker d'artefacts.
- **Blocage réseau interne (DNS Rebinding 403)** : Résolu par l'implémentation d'un patch d'interception HTTP surchargeant l'en-tête `Host`.
- **Conflits de Schéma SQL** : Résolu par la centralisation de la fonction d'initialisation de la base de données (Single Source of Truth). La non-régression et le bon déploiement de l'ensemble de ces solutions sont validés systématiquement par la chaîne d'intégration continue (CI) via GitHub Actions.